



SEEK - O - SPHERE

Technical

Documentation · Developer Guide



Architecture, file reference, data model, security design and the conventions that hold the Seek-O-Sphere codebase together.

Stack: vanilla HTML/CSS/JS · Backend: Firebase compat v10.12.0 · Project: ai-classroom-ad779

Updated 16 August 2026

1. Architecture

The single most important fact about this codebase: **index.html is not one page. It is eleven sections in one document.**

```
const ALL_PAGES = ['home', 'what', 'history', 'types', 'study', 'ethics',  
                  'lab', 'finder', 'resources', 'help', 'contact'];
```

`showPage(id)` adds `.active` to one panel and removes it from the rest. Two consequences follow, and each has already caused a real bug.

- **Anything scoped "per page" must be scoped per section.** Turning a feature off "for this file" turns it off for ten sections in order to affect one.
- **A check that only opens the home page will pass while the site is broken.** Content that escapes a panel is invisible on home — where it belongs — and appears underneath every other section, because "show one panel at a time" cannot hide what is outside all of them.

Everything else under `pages/` is either an authenticated page (login, dashboard, admin, account-settings), a real content page (mission), or a thin shell that mounts the shared nav and footer.

2. File Reference

Path	Role
<code>index.html</code>	The app. All eleven learning sections, the hero, the inline progress-writing logic.
<code>js/session.js</code>	Shared auth session and account menu. Paints from cache, reconciles with Firestore.
<code>js/core/role.js</code>	Who is signed in and where they belong. Remembers the role per tab as well as per device.
<code>js/core/theme.js</code>	Pins <code>data-theme="dark"</code> and clears the old preference. No longer a switch.
<code>js/core/planner.js</code>	The study planner panel: day, week and month views.
<code>js/core/nav.js</code> · <code>footer.js</code>	Shared chrome mounted by every standalone page.
<code>js/components/eulid.js</code>	The mascot: one copy, drag, five tool legs, position memory.
<code>js/components/lab.js</code> · <code>finder.js</code> · <code>marquee.js</code> · <code>backdrop.js</code>	Feature modules.
<code>js/questions-engine.js</code>	Local and optional Claude-API question engine behind one switch.
<code>css/theme-dark.css</code>	The Observatory palette. The live look, applied through tokens.
<code>css/dashboard.css</code>	Login, dashboard and admin styling.
<code>firestore.rules</code>	Access control, including the school boundary.
<code>structure.js</code>	Comparative page-shape check. See section 7.

3. Design Tokens and Type

The site paints almost everything through `--white`, `--text`, `--muted`, `--border` and their siblings. That is what let a single token block in `theme-dark.css` convert the whole site to dark without touching the rules that use them. **Write new CSS through tokens so it converts too.**

Typefaces have fixed roles: Space Grotesk for display, Inter for reading, Nunito for the marquee, JetBrains Mono for small uppercase labels. A mascot speech box should be Space Grotesk 500 at 1rem, line-height 1.5, letter-spacing -.01em, in `#E6EDF6`, capped near 19rem so lines break at roughly 30–35 characters.

4. Data Model

```
users/{uid}
  name, email, role ('student' | 'schooladmin' | 'owner'),
  schoolId, bucket ('school:<id>' | 'online' | 'staff'),
  curriculum, ageGroup, photo,
  totalQuestions, correctTotal, bestStreak, createdAt, lastActive

schools/{schoolId}      name, joinCode, createdAt      (owner-writable only)
joinCodes/{code}        the document ID is the code itself

progress/{uid}/daily/{YYYY-MM-DD__mode__subject}
  date, mode, subject, label, questions, correct, lastTs

planner/{uid}/...       per-user, per-date study planner
```

Every answered question increments the matching daily document and the user aggregate. Writes use `merge: true` and `FieldValue.increment`, so they are safe to repeat and work offline.

5. Security Design — the school boundary

Three roles: `owner` sees everything; `schooladmin` sees only their own school's students — no other school, no online students, and no admin accounts, not even their own row; `student` sees only themselves.

Firestore cannot filter a list for you. It can only refuse a query whose results it cannot guarantee are all permitted. So every user document carries a derived `bucket`:

```
'school:<schoolId>'  a student in that school
'online'             a student who joined without a code
'staff'              any admin or the owner
```

A school admin's panel queries `where('bucket','==','school:<their id>')`. Every document that query can return is by definition one of their own students, so the rule is satisfiable and the query is allowed. Ask for anything wider and Firestore refuses the whole query, not just the extra rows. `bucket` is recomputed by the write rules from `role` and `schoolId` and rejected if it disagrees, so nobody can edit their own profile to appear in — or vanish from — a school's list. A single equality filter is covered by Firestore's automatic index, so there is no composite index to create.

A limitation recorded honestly. Rules are all-or-nothing per document, and a planner day is one document, so an admin permitted to add an item is technically able to read that whole day. The panel therefore shows an admin only the items an admin set, and a student's own notes are round-tripped untouched and never displayed. That is a decision in the application code, not a guarantee from the rules.

6. Conventions

- **Firestore compat SDK only, from CDN script tags. Never ES modules.** The modular SDK caused silent failures. `js/firebase-config.js` holds an ES-module version for reference only.
- **Tokens, not literal colours.**

- **The site is dark only.** The light tokens survive as the layer the dark palette overrides.
- **Performance, measured here:** never put `filter: blur()` on an ancestor of something that animates — re-blurring every frame measured 6fps against 34. Never paint the same large gradient twice per frame. Split scroll offset and animation onto two nested elements.
- **The ambient looping animations are final** — the starfield and the mascot's idle motion are not to be changed.

7. Verification

`structure.js` renders a pristine baseline and the edited page side by side and asserts: the same panels in the same order, no panel nested inside another, nothing newly escaped its panel, no panel silently lost content, and `<div>` tags balanced.

```
node structure.js
EXPECT_LOSS="-69" node structure.js    # declare an intended removal
```

Two design points are worth preserving. The baseline must be a copy taken *before* the session's edits — once a change ships, the file on disk *is* the edited page, and comparing it with itself passes for the wrong reason. And a panel losing content must be **declared**, not tolerated: a check that quietly forgives the one number an edit was meant to move forgives every other movement of that number too.

More generally: verify by rendering and reading pixels, not by reading tokens or markup. If a check passes on a version you know is broken, suspect the check.

8. Traps Already Hit

Trap	What happens
Removing a block by searching for the next <code></div></code>	It finds the tag closing a <i>nested</i> element, so one closing tag too few is removed, <code>#home</code> closes early, and everything after it appears on every other section. Use a depth-counting walker.
Testing only the home page	Site-wide breakage is invisible there. Test every section.
<code>visibility:hidden</code> when measuring contrast	It hides the element's own background too, so you measure the page behind a button instead of the button. Use <code>color: transparent</code> for anything that paints a fill.
Two copies of the mascot	Duplicate SVG gradient ids: every fill in the second silently resolves to the first.
<code>Range.surroundContents()</code>	Throws on most real selections. Walk and split text nodes; call <code>parent.normalize()</code> after removing a mark.
<code>speechSynthesis</code>	A read-only accessor on <code>window</code> ; stub it with <code>Object.defineProperty</code> .
Press versus drag	Needs pointer capture and a distance threshold, or every click is read as a drag.
<code>!important</code> as a trump card	It does not settle specificity. <code>!important</code> on <code>.panel.active .hero-content</code> outranks <code>!important</code> on <code>*</code> .
Navigating between URLs differing only by fragment	A same-document navigation: stale DOM attributes persist.
Splitting CSS values on commas	Shatters <code>cubic-bezier(...)</code> , and the check silently passes.